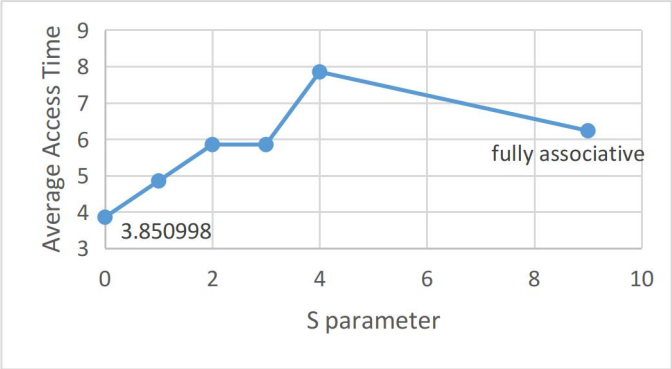
First, it is trivial to see that having a Victim Cache is better than not having one, because it improves the overall cache hit rate at no cost of access time. Therefore, we will conduct trials only for the configurations with 4 VC slots.

With the largest cache size 48KB, the maximum C is 15, which gives 32KB of data store. The maximum valid range of B should be between 1 and 6, inclusive, without exceeding the limit. I have conducted the following trials to observe trends.

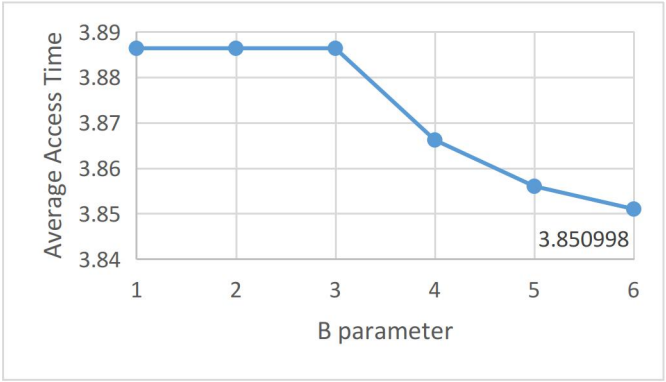
mm_basic_192.trace

1. Fix C=15, B=6, LRU and vary S.



As suggested by the chart, higher set-associativity does not decrease AAT in our case. It could be because that the data access pattern is sparse such that more blocks per set does not provide more hits in that set. The AAT is compromised because higher associativity has higher access time. This trend is observed from several other C, B values, and replacement policies.

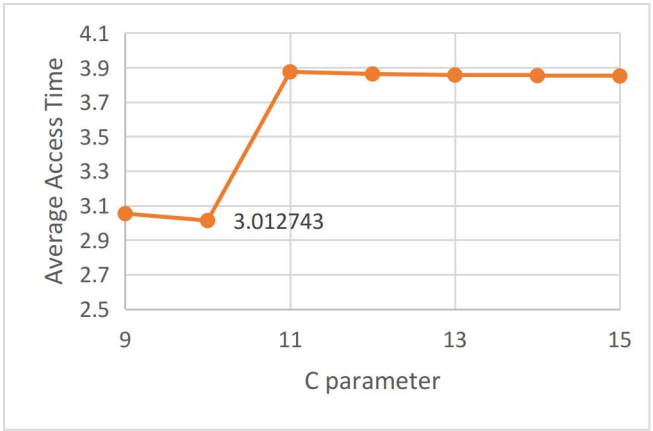
2. Fix C=15, S=0, LRU and vary B.



As suggested by the chart, higher block size helps improving AAT by introducing more spacial locality. This can be seen from other configurations as well.

Until now we have concluded direct-mapped cache with the highest block size gives the best AAT.

3. Fix B=6, S=0 and vary C. Replacement policies are irrelevant here since it's direct-mapped.



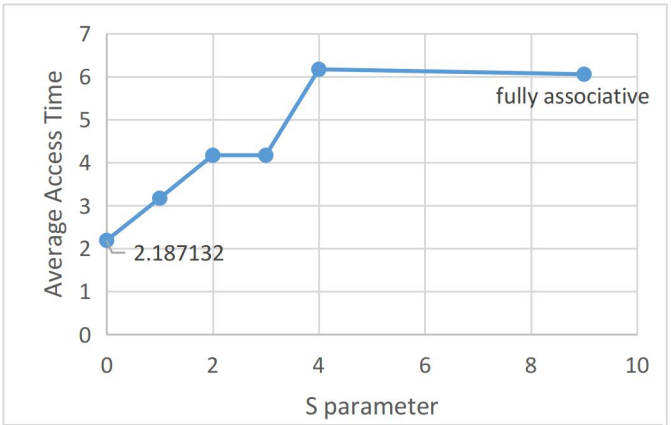
As suggested by the chart, C=10 provides the best AAT compared to other cache sizes. It is quite surprising to see, however, that a small cache in this case turned out to be the faster one to access. It might be because the algorithm is not taking advantage of a larger cache. In other words, accesses of the same block are too sparse for a cache hit to happen even with a large cache The longer access time of a bigger cache now jeopardizes the overall AAT.

Conclusion: for mm_basic_192.trace, the optimal configuration is C=10, B=6, S=0, irrelevant of replacement policy. This configuration is within the 48KB limit.

mm_tiled_192_4.trace

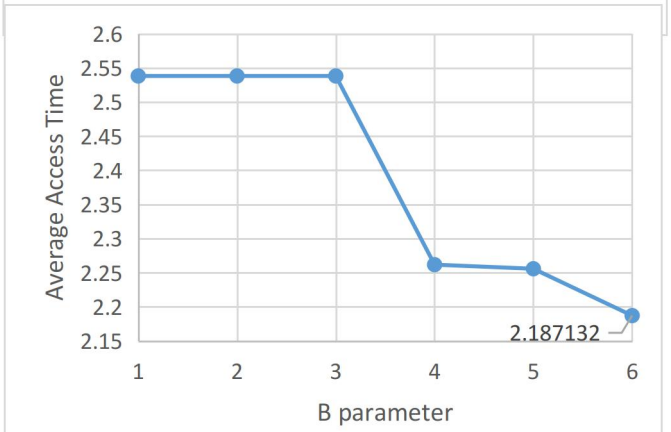
We will adhere to the same doctrine for the previous experimental trace.

1. Fix C=15, B=6, LRU and vary S.



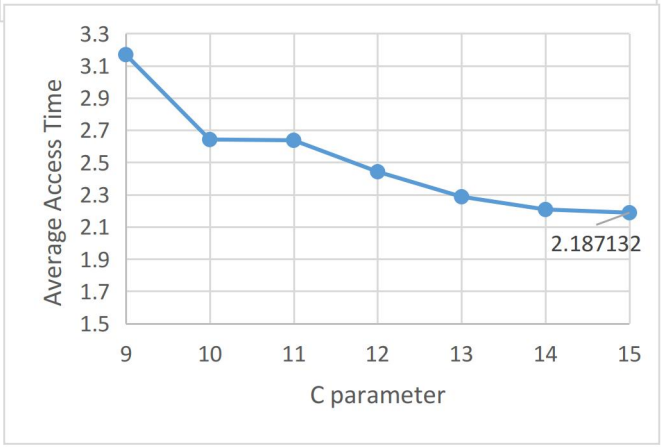
As suggested by the chart, higher set-associativity does not decrease AAT in our case. It could be because that the data access pattern is, while hitting the same index/set, sparse such that more blocks per set does translate to more hits in that set The AAT is heavily compromised because in our case higher associativity brings higher access time but not enough higher hit rate. This trend is observed from several other C, B values and is coherent with the previous experimental trace file.

2. Fix C=15, S=0, LRU and vary B.



As suggested by the chart, higher block size helps improving AAT by introducing more spacial locality. This can be seen from other configurations as well and matches with the previous experimental trace.

3. Fix B=6, S=0 and vary C. Again, replacement policies are irrelevant here since it's direct-mapped.



Different from the previous experimental trace, mm_tiled_192_4 prefers the largest cache size, C=15. This illustrates the advantage of the loop unrolling technique such that loaded values are used many times per iteration. As seen, this technique can take advantage of a larger cache size and is considerably quicker (~1.5x) than the basic loop.

Conclusion: for mm_tiled_192_4.trace, the optimal config is C=15, B=6, S=0, irrelevant of replacement policy. This gives a total cache size ~35KB.

Note: these two conclusions are confirmed thereafter by brute-forcing through all valid configurations.